ApproveHub

Digital document approval with tamper-evident digital approvals

Document	Final project information (all decisions in one place)
Course / project	EDI1 (ES26102), FY B.Tech IT, VIT Pune
Team	Group 9: Shreeyash (team leader), Shravani, Shreya Sutar, Shreya Loya
Status	Step 4 (database design) complete and ready for team sign-off. Date: 4 November 2026
Unlocks	Step 5 (API list) and Step 8 (database models)

How to read this document. Sections 1 to 4 explain what the system does. Section 5 is the final database design (the Step 4 deliverable). Sections 6 to 9 explain how approvals stay secure, the problems we expect and how we solve them, and the low-effort improvements that raise the quality of the result. Sections 10 to 14 give the draft API list, screens, tools, the sign-off checklist and a glossary of every term for beginners.

Important rule for the whole team: table and column names in Section 5 are locked once everyone signs off. Nobody renames a column without telling the group, because the frontend, backend and database code all depend on the same names.

1. Project overview

The problem. Approvals on paper are slow: a student or staff member must walk a document from a professor to the HOD and onwards. Signatures and stamps are easy to forge or copy, nobody can see where a document is stuck, and a printed copy cannot be verified later.
The solution. ApproveHub is a web and mobile app. The applicant uploads a PDF, chooses the approvers in order and drags one approval mark per approver to a position on the page. Each approver reviews the document and approves with a PIN. The server creates the approval marks itself, stamps them on the PDF, adds a QR code and a unique approval number, and stores a digital signature (a cryptographic fingerprint) for every approval. Anyone can scan the QR code or type the approval number to verify the document online.

Goals

- Replace paper approvals with a clear, ordered, trackable online chain.
- Make approvals hard to forge: no uploaded signature images, only server-generated marks backed by digital signatures and hashes.
- Work for many institutions at once, with an admin who builds the hierarchy and grants or withdraws access.
- Let anyone verify a printed or PDF document using its QR code or approval number.
- Keep the project small enough for the team to finish: reuse the same few building blocks (hashing, one PDF stamping function, one verification endpoint) everywhere.

Out of scope (say this clearly in the report)

- Legally certified e-signatures such as Aadhaar eSign or a Digital Signature Certificate. ApproveHub is a tamper-evident institutional approval workflow, not a certified signature service.
- Email or SMS notifications (we use in-app notifications), OCR, offline mode, editing the PDF content inside the app.

2. Roles and permissions

Role	Can do	Cannot do
Applicant	Sign up and log in; upload a PDF; choose approvers in order; drag a mark per approver; submit; track status and timeline; download the final PDF; fix and resubmit when a document is returned (compliance); cancel own pending document.	Approve anything; see other people's documents; upload or use signature, stamp or seal.
Approver	Everything an applicant can do (they can also submit their own documents) plus: set and use a PIN; see the inbox of requests waiting for them; approve, return for compliance, or reject with a remark; see their own approved documents; keep signature, stamp and seal in their profile.	Act on a step that is not pending for them; act without the correct PIN; act while the account is locked or inactive.
Admin	Manage users, roles and access (grant or withdraw); build the institution hierarchy; see the list of all documents; search by approval number and open the exact file; reassign a pending step if an approver leaves; see dashboard totals.	Approve on behalf of someone; edit a document file or an approval record.
Public verifier (no login)	Open the verification page from the QR code or approval number; upload a PDF to check it matches the stored fingerprint.	See the document content or any personal data beyond what the verification page shows.

3. End-to-end workflow

A. First use

- Open the app and sign up or log in. After the first login, the user sets a PIN (mandatory for approvers; recommended for everyone as an app lock).
- The admin assigns the user's institution, role, post and department.

B. Applicant submits a document

- Upload a PDF. The server checks that it is a real, unencrypted PDF under the size limit, computes **original_hash** (SHA-256), counts the pages and creates a random **approval_no**.
- Pick branch or department, designation and the approver for each step, in order. Add more approvers (for example Professor, then HOD).

- Drag one mark per approver on a page preview. The app suggests default positions at the bottom of the last page, so the applicant only adjusts if needed. The browser sends the positions as fractions (0 to 1) of the page size.
- Submit. The document becomes **pending**, step 1 becomes **pending**, other steps stay **waiting**, a timeline entry is written and approver 1 receives a notification.

C. Approver acts

- The approver opens the inbox, reads the document and chooses **Approve**, **Return for compliance** or **Reject** (the last two need a remark).
- For every action the approver enters the PIN. Wrong PINs are counted and the account locks for a few minutes after repeated failures.
- On approve, the server sets acted_at (server time), creates this approver's mark at the saved position, stores the digital signature in **record_hash** and moves the next step from waiting to pending.

D. Completion

- After the last approval the server builds the final PDF from the untouched original plus all marks, adds the QR code at the top right and the approval number, then stores **final_path** and **final_hash**. The document becomes **approved** and the applicant is notified.
- If any approver rejects, the document becomes **rejected**. If an approver returns it for compliance, the applicant corrects the file and resubmits (see Section 6).

E. Verification (anyone)

- Scan the QR code or type the approval number: the verification page shows status, each approver, time of approval and whether every digital signature is valid.
- Optionally upload a PDF: the server hashes it and tells the user whether it matches the stored final_hash.

4. Feature list with priority

Must = needed for a working demo. Should = clearly improves the result. Could = only if time remains. Effort: S = hours, M = about a day, L = several days.

Feature	Role	Priority	Effort
Sign up, login, PIN setup	All	Must	M
Upload PDF, choose approvers, order of steps	Applicant	Must	M
Drag-and-place a mark per approver (stored as fractions of the page)	Applicant	Must	M
Approver inbox; approve / return / reject with PIN	Approver	Must	M
Server-side stamping of marks, QR code and approval number on the final PDF	System	Must	L
Digital signature per approval (record_hash) and file fingerprints (original_hash, final_hash)	System	Must	S
Public verification page (QR code / approval number)	Public	Must	S
Timeline (history) page for each document	All	Must	S
Admin: all documents list, search by approval number, open the file	Admin	Must	S
Admin: users, roles, grant or withdraw access (is_active)	Admin	Should	S
Admin: hierarchy builder (hierarchy_links)	Admin	Should	M
Upload-a-PDF-to-verify tamper check	Public	Should	S
Dashboard counts (pending, approved, rejected)	All	Should	S
In-app notifications (bell icon, unread count)	All	Should	S
PIN lockout after wrong attempts	Approver	Should	S
Suggested default mark positions	Applicant	Should	S
Highlight requests pending for more than N days	Approver / Admin	Could	S
Demo data seeding script	Team	Should	S
Saved approver chains (templates), audit-report PDF download	Applicant / Admin	Could	M

5. Final database design (Step 4 deliverable)

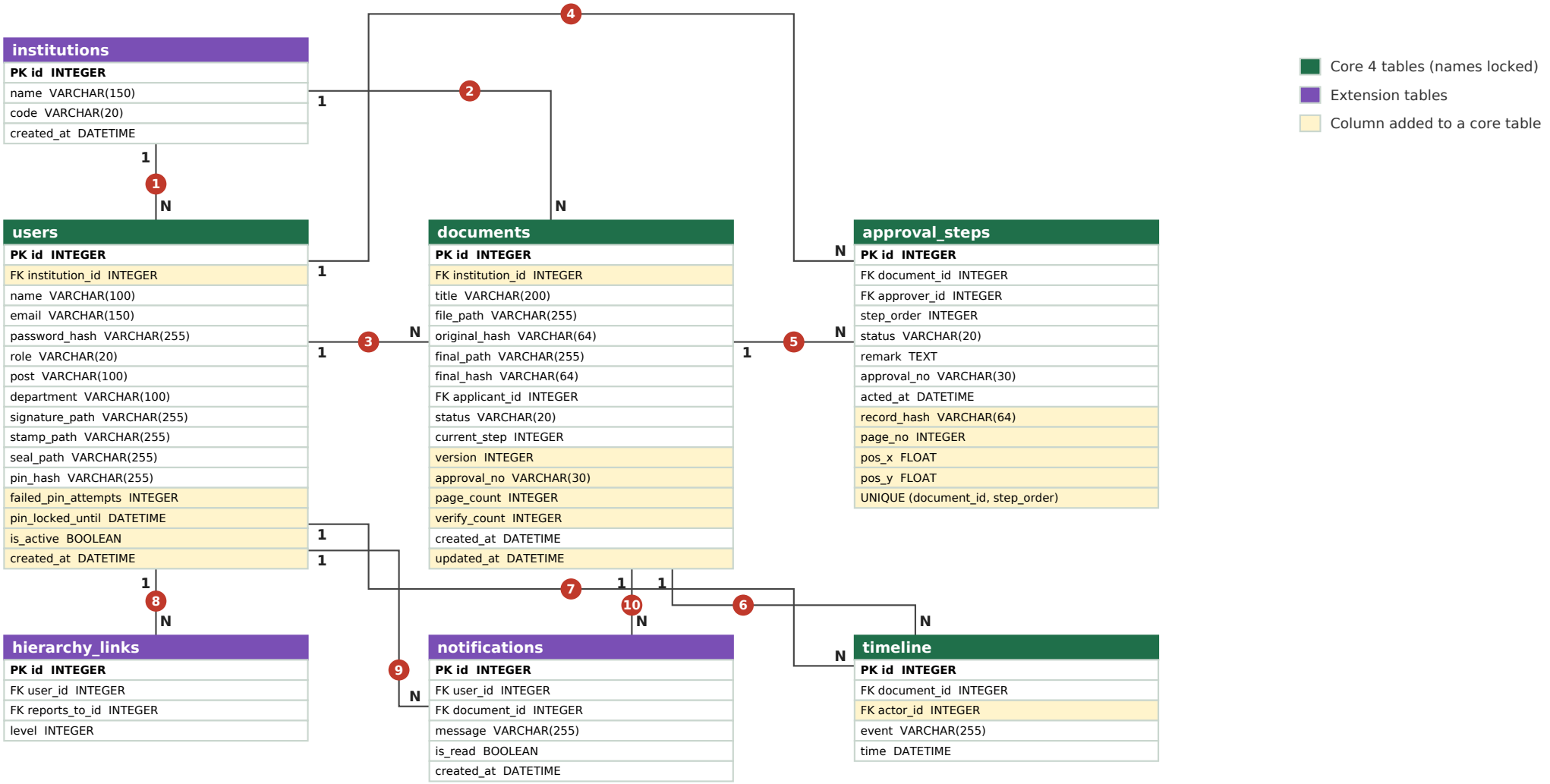
5.1 Ideas to understand first

- **Table** = one Excel sheet. **Row** = one record. **Column** = one property.
- **Primary key (PK)** = the unique *id* of each row. **Foreign key (FK)** = a column that stores the id of a row in another table, so data is linked instead of copied.
- **One-to-many**: one user has many documents; one document has many approval steps, timeline events and notifications.
- **Nullable** = the column may be empty (for example final_path is empty until the document is approved).
- **Cascade rules** decide what happens to child rows when a parent row is deleted (explained in 5.4).

5.2 What is locked and what is added

Item	Status	Notes
Tables users, documents, approval_steps, timeline and all their original columns	**Locked**	Exactly as in the blueprint. Only data types and rules were added.
Added columns on the core tables (yellow rows in the diagram)	Proposed	institution_id, is_active, created_at, PIN lockout fields; document version, approval_no, page_count, verify_count, updated_at; step record_hash, page_no, pos_x, pos_y; timeline actor_id.
Added tables institutions, hierarchy_links, notifications	Proposed	Support multi-institution use, the admin hierarchy builder and in-app notifications.
signature_path, stamp_path, seal_path	**Team decision**	Kept for approvers only, as a profile record. The server draws marks itself, so these files are never printed on documents. If nobody needs them, drop all three before Step 8.

ApproveHub — final ER diagram (1 = parent side, N = child side; numbers match the relationship list below)



1 institutions 1 to N users (institution_id), on delete: **SET NULL**
2 institutions 1 to N documents (institution_id), on delete: **RESTRICT**
3 users 1 to N documents (applicant_id), on delete: **RESTRICT**
4 users 1 to N approval_steps (approver_id), on delete: **RESTRICT**
5 documents 1 to N approval_steps (document_id), on delete: **CASCADE**
6 documents 1 to N timeline (document_id), on delete: **CASCADE**
7 users 1 to N timeline (actor_id), on delete: **SET NULL**
8 users 1 to N hierarchy_links (user_id and reports_to_id), on delete: **CASCADE**
9 users 1 to N notifications (user_id), on delete: **CASCADE**
10 documents 1 to N notifications (document_id), on delete: **CASCADE**

5.3 Table-by-table details

Yellow rows mark columns or tables added beyond the original blueprint. Data types are written generically; SQLAlchemy in Step 8 will map them (Integer, String, Text, Float, Boolean, DateTime).

users (core table)

Every person who logs in: applicant, approver or admin.

Key	Column	Type	Rules / notes
PK	id	INTEGER	auto-increment
FK	institution_id	INTEGER	NULL -> institutions.id, SET NULL on delete
	name	VARCHAR(100)	NOT NULL
	email	VARCHAR(150)	UNIQUE, NOT NULL
	password_hash	VARCHAR(255)	NOT NULL (bcrypt/argon2 hash, never the password)
	role	VARCHAR(20)	NOT NULL: applicant | approver | admin
	post	VARCHAR(100)	NULL (designation, e.g. Professor, HOD)
	department	VARCHAR(100)	NULL
	signature_path	VARCHAR(255)	NULL, approvers only (profile record; not drawn on documents)
	stamp_path	VARCHAR(255)	NULL, approvers only
	seal_path	VARCHAR(255)	NULL, approvers only
	pin_hash	VARCHAR(255)	NULL until PIN is set; mandatory for approvers (hash, never the PIN)
	failed_pin_attempts	INTEGER	default 0 (PIN lockout counter)
	pin_locked_until	DATETIME	NULL (set after too many wrong PINs)
	is_active	BOOLEAN	default true (admin can withdraw access)
	created_at	DATETIME	default now (UTC)

documents (core table)

Every uploaded document and its overall approval status.

Key	Column	Type	Rules / notes
PK	id	INTEGER	auto-increment
FK	institution_id	INTEGER	NULL -> institutions.id, RESTRICT
	title	VARCHAR(200)	NOT NULL
	file_path	VARCHAR(255)	NOT NULL (original PDF, stored outside the public folder)
	original_hash	VARCHAR(64)	NOT NULL (SHA-256 of the uploaded file)
	final_path	VARCHAR(255)	NULL until fully approved (stamped PDF with QR)
	final_hash	VARCHAR(64)	NULL until fully approved (SHA-256 of final PDF)
FK	applicant_id	INTEGER	NOT NULL -> users.id, RESTRICT
	status	VARCHAR(20)	NOT NULL: pending | compliance | approved | rejected | cancelled
	current_step	INTEGER	default 1 (step_order currently waiting for action)
	version	INTEGER	default 1 (increases on each resubmission)
	approval_no	VARCHAR(30)	UNIQUE, NOT NULL (random, e.g. AH-2026-7K3QX9)
	page_count	INTEGER	NOT NULL (to validate mark page numbers)
	verify_count	INTEGER	default 0 (times the QR/verification page was opened)
	created_at	DATETIME	default now (UTC)
	updated_at	DATETIME	default now, updated on change

approval_steps (core table)

One row per approver per document: the approval chain, with the mark position.

Key	Column	Type	Rules / notes
PK	id	INTEGER	auto-increment
FK	document_id	INTEGER	NOT NULL -> documents.id, CASCADE
FK	approver_id	INTEGER	NOT NULL -> users.id (role must be approver), RESTRICT
	step_order	INTEGER	NOT NULL (1, 2, 3 ...)
	status	VARCHAR(20)	NOT NULL: waiting | pending | approved | compliance | rejected
	remark	TEXT	NULL (approver's comment / reason)
	approval_no	VARCHAR(30)	NULL until approved (short code shown inside this approver's mark)
	acted_at	DATETIME	NULL until acted (server time, UTC)
	record_hash	VARCHAR(64)	NULL until approved (HMAC digital signature of this approval)
	page_no	INTEGER	default 1 (page where the mark is placed)
	pos_x	FLOAT	0 to 1 (fraction of page width)
	pos_y	FLOAT	0 to 1 (fraction of page height)
	UNIQUE	(document_id, step_order)	no two steps share the same order

timeline (core table)

Permanent audit trail: every event on a document, in order.

Key	Column	Type	Rules / notes
PK	id	INTEGER	auto-increment
FK	document_id	INTEGER	NOT NULL -> documents.id, CASCADE
FK	actor_id	INTEGER	NULL -> users.id, SET NULL (who did it)
	event	VARCHAR(255)	NOT NULL (e.g. 'Approved by HOD')
	time	DATETIME	default now (UTC)

institutions (extension table)

One row per college/organisation using ApproveHub (multi-institution support).

Key	Column	Type	Rules / notes
PK	id	INTEGER	auto-increment
	name	VARCHAR(150)	NOT NULL
	code	VARCHAR(20)	UNIQUE, NOT NULL (short code, e.g. VITP)
	created_at	DATETIME	default now (UTC)

hierarchy_links (extension table)

Who reports to whom inside an institution (built by the admin).

Key	Column	Type	Rules / notes
PK	id	INTEGER	auto-increment
FK	user_id	INTEGER	NOT NULL -> users.id, CASCADE
FK	reports_to_id	INTEGER	NULL -> users.id, CASCADE
	level	INTEGER	NOT NULL (rank in the hierarchy)

notifications (extension table)

In-app notifications (bell icon): new request, approved, returned.

Key	Column	Type	Rules / notes
PK	id	INTEGER	auto-increment
FK	user_id	INTEGER	NOT NULL -> users.id, CASCADE
FK	document_id	INTEGER	NULL -> documents.id, CASCADE
	message	VARCHAR(255)	NOT NULL
	is_read	BOOLEAN	default false
	created_at	DATETIME	default now (UTC)

5.4 Relationships and cascade rules

CASCADE = deleting the parent also deletes its children. **RESTRICT** = deleting the parent is blocked while children exist. **SET NULL** = the child stays and its pointer becomes empty. In practice documents are almost never hard-deleted: users are deactivated with is_active, and applicants cancel documents (status cancelled).

#	Parent (one)	Child (many)	Column	On parent delete	Why
1	institutions	users	institution_id	SET NULL	A user may outlive an institution record
2	institutions	documents	institution_id	RESTRICT	Never lose documents by deleting an institution
3	users	documents	applicant_id	RESTRICT	Never lose a user's documents
4	users	approval_steps	approver_id	RESTRICT	Never lose approval history of an approver
5	documents	approval_steps	document_id	CASCADE	Steps are meaningless without their document
6	documents	timeline	document_id	CASCADE	Timeline belongs to one document
7	users	timeline	actor_id	SET NULL	Keep history even if the actor is removed
8	users	hierarchy_links	user_id and reports_to_id	CASCADE	Hierarchy rows belong to the user
9	users	notifications	user_id	CASCADE	Notifications belong to the user
10	documents	notifications	document_id	CASCADE	Notifications belong to the document

5.5 Constraints and indexes

- **UNIQUE**: users.email; documents.approval_no; approval_steps (document_id, step_order); institutions.code.
- **CHECK**: users.role and the two status columns only accept the allowed values; approval_steps.pos_x and pos_y are between 0 and 1.
- **Indexes** (make searching fast): documents.approval_no, documents.applicant_id, documents.status; approval_steps (approver_id, status) for the approver inbox; timeline (document_id, time); notifications (user_id, is_read).
- **Always use UTC server time** for every date-time column. Never trust the browser's clock.

6. Status rules (state flow)

Step status (approval_steps.status)

Status	Meaning	Next status
waiting	An earlier step is not finished yet.	pending (when the previous step is approved)
pending	This approver must act now.	approved, compliance or rejected
approved	Approved with PIN; digital signature saved.	final
compliance	Approver returned the document with a remark; the applicant must fix it.	reset to waiting / pending when the applicant resubmits
rejected	Approver rejected the document with a remark.	final

Document status (documents.status)

Status	Meaning	How it changes
pending	Submitted; the approval chain is running (current_step shows where).	approved when the last step is approved; compliance, rejected or cancelled otherwise
compliance	Returned to the applicant for corrections.	pending again after resubmission
approved	All steps approved; final PDF with QR exists.	final
rejected	An approver rejected it.	final (applicant may upload a new document)
cancelled	Applicant cancelled while pending.	final

Rules to code exactly once, in one place in the backend

- Only the approver of the **pending** step may act, and only with the correct PIN.
- On approve: mark step approved, set acted_at, store record_hash, make the next waiting step pending and update documents.current_step. If it was the last step: build the final PDF, set final_path, final_hash and status approved.
- On compliance: step status compliance, document status compliance. On resubmission the applicant replaces the file: version increases by 1, original_hash is recomputed, **all earlier approvals are cleared** (they referred to different content), step 1 becomes pending again and a timeline entry records the reset.
- Every change writes a timeline row with actor_id. Do the status change and the timeline row in one database transaction so they cannot disagree.

7. Digital approval mark and security design

The visible mark is only a picture. The proof is stored on the server. A copied or drawn tick has no matching record, so verification fails.

Part	What it is	Where it lives
Approval mark	A green tick box drawn by the **server** at the saved position. It shows the approver's name, designation, date and a short code. The applicant never supplies an image.	Stamped into the final PDF only
approval_no (document)	Random unique number such as AH-2026-7K3QX9, printed with the QR code. Generated with a secure random function (not a counter), so numbers cannot be guessed.	documents.approval_no
Step code	Short code for one approver's approval, shown inside that approver's mark.	approval_steps.approval_no
original_hash	SHA-256 fingerprint of the uploaded file. Changes if even one byte changes.	documents.original_hash
record_hash (digital signature)	HMAC-SHA256 of the approval details using a secret key that exists only on the server: document_id | version | original_hash | approver_id | step_order | status | acted_at. Nobody without the key can create or alter a valid record.	approval_steps.record_hash
final_hash	SHA-256 fingerprint of the final stamped PDF.	documents.final_hash
QR code	Encodes the verification link with the approval number, placed at the top right of the final PDF.	Stamped into the final PDF

Verification algorithm (public page)

- Look up the document by approval_no. If not found, show "not found" (do not reveal why).
- Recompute the HMAC for every approved step and compare with record_hash. Any mismatch means the database row was altered.
- If a PDF is uploaded, compute its SHA-256 and compare with final_hash. Match means the file is unchanged since approval.
- Show status, each approver (name, designation), time of approval and the two checks. Increase verify_count. Limit how often one visitor may call the endpoint, to stop number guessing.

PIN and secret handling

- Store only **hashes** of passwords and PINs (bcrypt or argon2). A 4-digit PIN has only 10,000 possibilities, so a leaked hash could be broken quickly; therefore use a **6-digit PIN** and the lockout (failed_pin_attempts,

pin_locked_until).

- Keep the HMAC secret key and database credentials in an **.env file** that is listed in .gitignore. Never commit it to GitHub.
- Serve uploaded and final files only through an endpoint that checks the logged-in user's permission. Never put them in a public folder.

8. Problems we expect and their solutions

#	Problem	Solution
1	Fake mark: someone draws or copies a tick in a PDF editor.	The proof is the server record. The mark carries a code and the QR code opens the verification page; a fake has no valid record.
2	Screen position does not match PDF position (pixels versus points, different origin, zoom, page sizes).	Store pos_x and pos_y as fractions 0 to 1 of the page width and height. At stamping time multiply by the real page size and flip the vertical axis if the PDF library needs it.
3	Mark covers text, sits outside the page or overlaps another mark.	Validate on the backend that the whole mark is inside the page and warn on overlap. Show a preview before submit. Offer default positions.
4	The file is edited after approval.	Original is read-only. Compare hashes on verification (original_hash, final_hash).
5	Document changes while approvals are in progress.	No editing after submission. A returned document is replaced by resubmission, which resets all approvals and increases version.
6	Stolen password or guessed PIN.	PIN required for every approval, 6 digits, lockout after repeated failures, hashed storage.
7	Stamping several approvers one by one corrupts or degrades the PDF.	Never modify the original. Build the stamped copy from the original plus all approved marks, and save final_path only after the last approval.
8	Dragging is imprecise on phones.	Large drag handle, snap to a grid, preview with zoom, default positions.
9	Unsafe or huge uploads; encrypted or already-signed PDFs.	Accept PDF only, check the real file type, set size and page limits (for example 10 MB, 30 pages), reject password-protected PDFs, store outside the public folder.
10	Unclear status flow between waiting, pending, approved, compliance.	Follow the tables in Section 6 and code them in one function.
11	Time manipulation.	Use server UTC time only.
12	Legal validity questions.	State clearly that this is a tamper-evident institutional approval system, not a certified e-signature.
13	Data leaks between institutions.	Every query filters by the logged-in user's institution_id (except a super admin). Test this explicitly.
14	An approver leaves or loses access while a step is pending.	Admin can reassign the pending step to another approver; the change is written to the timeline.
15	Secrets or passwords pushed to GitHub.	Use .env and .gitignore from day one; review every pull request for accidental secrets.
16	Guessable approval numbers or abuse of the verification page.	Random numbers, UNIQUE constraint, rate limit on the verification endpoint.
17	Double approval when the button is clicked twice.	Check that the step is still pending inside the same database transaction; the second request fails.
18	Slow stamping of very large PDFs.	File size and page limits; stamp only the pages that carry marks and the QR page where the library allows.
19	The QR link does not open on another phone during the demo (localhost).	Deploy to a public host or use the laptop's network address for the demo and test the QR code on a different phone beforehand.
20	Team members use different field names.	This document and the draw.io file are the single source of truth; changes go through a pull request.

9. Low-effort improvements with the biggest impact

Each of these reuses code you already need (hashing, one verification endpoint, the timeline table), so the extra work is small.

Improvement	Impact	Effort	Why it is worth it
Public verification page via QR code	Very high	S	Shows the whole idea working in one screen; mostly reads data you already store.

Improvement	Impact	Effort	Why it is worth it
Upload-to-verify tamper check	Very high	S	A tampered PDF turning red is a memorable demo moment; reuses the hash function.
Server-generated mark with name, designation, date and code	High	S	Looks professional and removes any need for uploaded images.
Suggested default mark positions	High	S	Fewer bad placements and a faster upload flow.
Timeline page per document	High	S	The timeline table already exists; only a display is needed.
Dashboard counts	High	S	Simple counting queries make the app look complete.
In-app notifications	High	S	One small table and a bell icon; no email setup.
PIN lockout	Medium	S	Strong security story with two small columns.
Random approval numbers	Medium	S	A few lines of code; prevents guessing.
Pending-too-long highlight	Medium	S	Compare dates and colour the row red; no schedulers needed.
QR scan counter (verify_count)	Medium	S	Admin can see how often documents are verified.
Demo data seeding script	High	S	One command creates an institution, users, sample documents; saves hours before the viva.
Saved approver chains; audit-report PDF	Medium	M	Only if time remains after all Must and Should items.

Demo checklist: use a seeded institution with an applicant, two approvers and an admin; show one approval end to end; scan the QR code on a phone; upload a modified copy of the final PDF to show the tamper warning; open the timeline and the admin search.

10. Draft API list (input for Step 5)

Proposal only. Finalise the exact paths and request/response shapes in Step 5 and test each one in Postman.

Method and path	Who	Purpose
POST /auth/register, POST /auth/login	All	Create account, log in (returns a token)
POST /auth/pin/set, POST /auth/pin/verify	Logged in	Set the PIN; check it before sensitive actions
GET /me	Logged in	Own profile, role, institution
POST /documents	Applicant	Upload PDF; returns document id and approval_no
PUT /documents/{id}/steps	Applicant	Set approvers, order and mark positions (page_no, pos_x, pos_y)
POST /documents/{id}/submit	Applicant	Start the approval chain
POST /documents/{id}/resubmit	Applicant	Replace the file after compliance; resets approvals
POST /documents/{id}/cancel	Applicant	Cancel a pending document
GET /documents, GET /documents/{id}	Applicant	My documents; one document with steps and timeline
GET /documents/{id}/download	Owner / approver / admin	Download original or final PDF (permission checked)
GET /approvals/inbox	Approver	Steps waiting for me
POST /approvals/{step_id}/approve (PIN)	Approver	Approve; stamp; sign; advance the chain
POST /approvals/{step_id}/compliance, /reject (PIN, remark)	Approver	Return or reject with a remark
GET /approvals/mine	Approver	My approved documents
GET /admin/documents?approval_no=&status=	Admin	List and search all documents
POST /admin/users/{id}/access, PUT /admin/users/{id}/role	Admin	Grant or withdraw access, set role
POST /admin/steps/{step_id}/reassign	Admin	Replace the approver of a pending step
GET/POST/PUT/DELETE /admin/hierarchy	Admin	Manage hierarchy_links
GET /verify/{approval_no}	Public	Verification data and signature checks
POST /verify/upload	Public	Upload a PDF and compare with final_hash
GET /notifications, POST /notifications/{id}/read	Logged in	Bell icon list and mark as read

Method and path	Who	Purpose
GET /dashboard/stats	Logged in	Counts for the dashboard

11. Screens (website and app)

Screen	Role	Main content
Sign up / Login	All	Email and password; first-time PIN setup
Dashboard	All	Counts, recent documents, notifications
Upload document	Applicant	PDF upload with validation messages
Choose approvers	Applicant	Department, designation, approver names, order; add more
Place marks	Applicant	Page preview, one draggable block per approver, default positions, overlap warning
My documents / document detail	Applicant	Status, current step, timeline, download, resubmit or cancel
Approver inbox	Approver	Pending requests, highlight if pending too long
Review and approve	Approver	Document preview, remark box, approve / return / reject with PIN
My approved documents	Approver	Searchable list with approval numbers
Admin: all documents	Admin	List, search by approval number, open file
Admin: users and access	Admin	Roles, activate or deactivate, reassign steps
Admin: hierarchy builder	Admin	Tree of who reports to whom
Verification page	Public	Result of QR / approval number check; upload-to-verify
Notifications and profile	All	Bell list; approver profile (signature, stamp, seal records)

12. Suggested building blocks

Choose the exact libraries together in Step 5 and Step 8. These are common options that match the Step 4 reference links (SQLAlchemy 2.0, Postman).

Need	Common option
Database models and relationships	SQLAlchemy 2.0 (unified tutorial linked in the blueprint)
Password and PIN hashing	bcrypt or argon2 libraries
File hashing and digital signature	Python hashlib (SHA-256) and hmac (HMAC-SHA256)
Random approval numbers	Python secrets module
QR code image	qrcode library
Stamping marks on a PDF	PyMuPDF, or reportlab combined with pypdf
Page preview in the browser	PDF.js, drawing a draggable block above the page canvas
Testing the API	Postman
Diagrams	draw.io (the .drawio file is the editable source)

13. Step 4 sign-off checklist and next steps

Definition of done for Step 4

- ER diagram PNG and the .drawio file are saved in docs/architecture/ on a feature branch (never push to main directly).
- Pull request opened and Shreeyash assigned to review and merge before the Week 1 deadline.
- Team has read this document and replied "agreed" in the group; field names are now locked.

Decisions the team must confirm (write the answers in the pull request)

- Keep or drop signature_path, stamp_path and seal_path? (recommendation: drop if no one has a use for them)
- Is one fixed mark design (green tick box) enough? (recommendation: yes)
- Is the PIN mandatory for approvers only, or for everyone? (recommendation: mandatory for approvers, optional app lock for others)
- Add the rejected status value for steps and documents? (recommendation: yes)

- Which extras are in scope: institutions, hierarchy_links, notifications? (recommendation: all three, because each is small)

Next steps

- **Step 5**: finalise the API list in Section 10 and test it in Postman.
- **Step 8**: write the SQLAlchemy models from Section 5.3, including constraints and indexes.
- Write the demo seeding script early so everyone tests with the same data.

14. Glossary for beginners

Term	Plain meaning
Database	The project's long-term memory, made of tables.
Primary key / Foreign key	A row's unique id / a column that points to a row in another table.
Nullable	The column is allowed to be empty.
Cascade / Restrict / Set null	What happens to child rows when the parent is deleted: delete them / block the delete / empty the pointer.
Index	A lookup shortcut that makes searching a column fast.
Transaction	A group of database changes that all succeed or all fail together.
Hash (SHA-256)	A fixed-length fingerprint of data. Any change to the data changes the fingerprint completely.
HMAC	A hash made with a secret key, so only someone with the key can produce a valid one. We use it as the digital signature of an approval.
Digital signature (in this project)	Cryptographic proof stored on the server that a specific approver approved a specific file at a specific time.
QR code	A scannable picture that holds the verification link.
Token	A temporary proof of login that the app sends with each request.
Rate limiting	Limiting how often a user may call an endpoint, to stop guessing and abuse.
Migration	A tracked change to the database structure (for example adding a column).